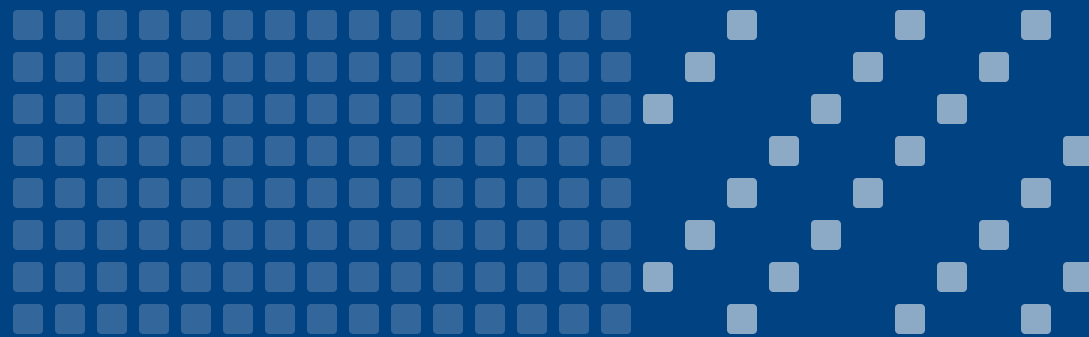




tokens → concepts

# VCM: Vision Concept Modeling

Adaptive Vision Token Compression via Instruction Fine-Tuning

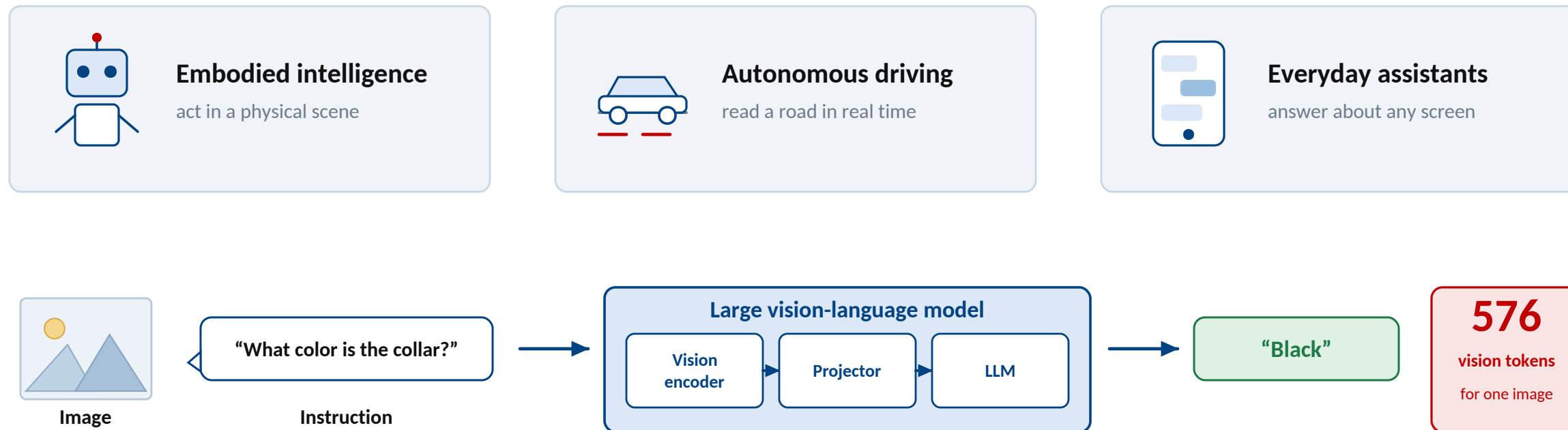
---

Run Luo · Renke Shan · Longze Chen · Ziqiang Liu · Lu Wang · Min Yang · Xiaobo Xia

SIAT, Chinese Academy of Sciences · UCAS · National University of Singapore · USTC

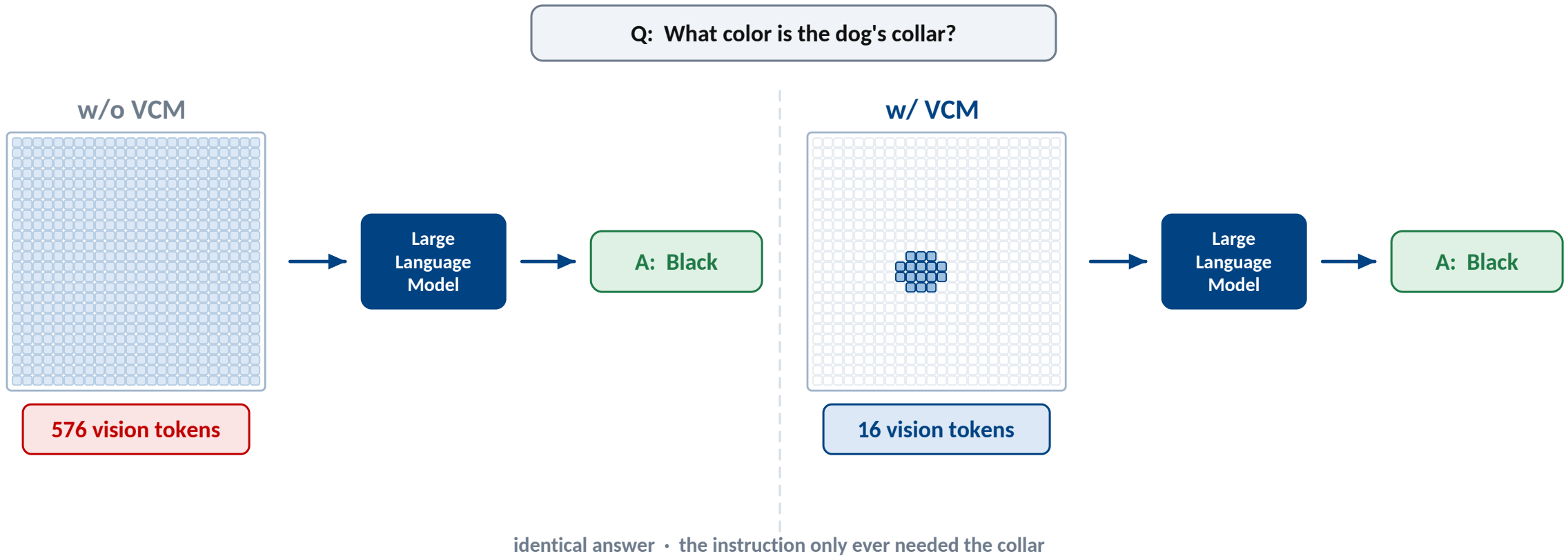
[arXiv:2504.19627v2](https://arxiv.org/abs/2504.19627) [cs.CL]

# LVLMs are the interface to the real world



**Powerful, general, deployed — and still reading every image token by token.**

# Humans answer with concepts, LVLMs pay for tokens



The question needs **a few concepts**; the model still pays for **all 576 tokens**.

# Why that redundancy is expensive

One LVLM forward pass, sequence length  $n$



Attention cost is quadratic in  $n$

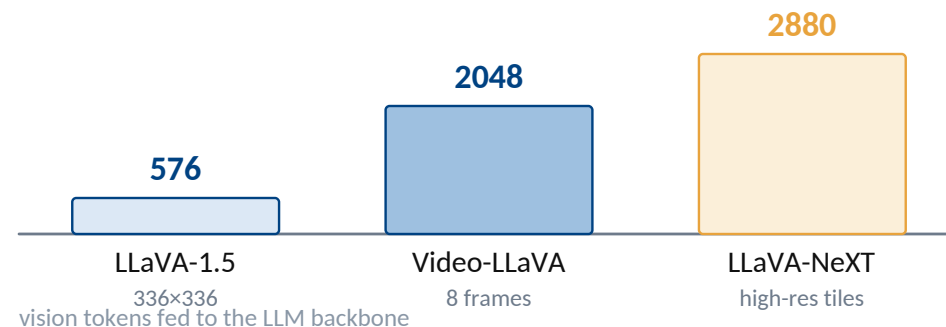
$$\text{FLOPs} = T \cdot (4nd^2 + \boxed{2n^2d} + 2ndm)$$

$\text{grows with } n^2$

Cutting the vision tokens is the only lever that touches the dominant term of the sequence.

$n$  scaled by  $1/8 \rightarrow R \approx 3/25 \rightarrow 85\%$  fewer FLOPs

... and the input keeps growing



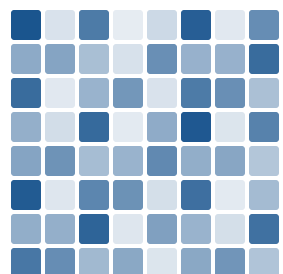
Vision tokens dominate the sequence, and attention is quadratic in its length.

Higher resolution and longer video make the **dominant** term grow **fastest**.

# The existing recipe: prune, or merge

## (a) Prune — threshold the attention matrix

attention matrix



top-k  
→



kept tokens keep their index,  
but carry no concept boundary

- no semantic concept, only surviving patches
- the number kept is set by a fixed ratio

## (b) Merge — fixed-length trainable queries



vision tokens



fixed-length learnable queries



always the same length,  
original order discarded

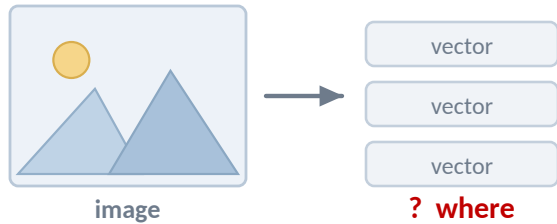
- relative order and position are lost
- length cannot follow the instruction

Both operate on tokens. Neither returns a concept that a downstream task can point at in the image.

Both make the sequence **shorter**; neither makes it **conceptual**.

# What is still missing

## No spatial grounding



compressed vectors do not say  
where in the image they came from

## Order and position lost

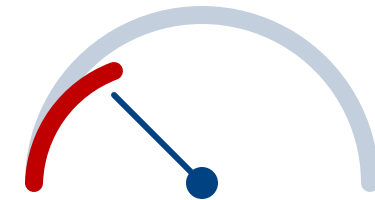


**order no longer recoverable**

→ region-level tasks break

merged queries drop the relative  
layout the answer may depend on

## Length not controllable



**fixed ratio**

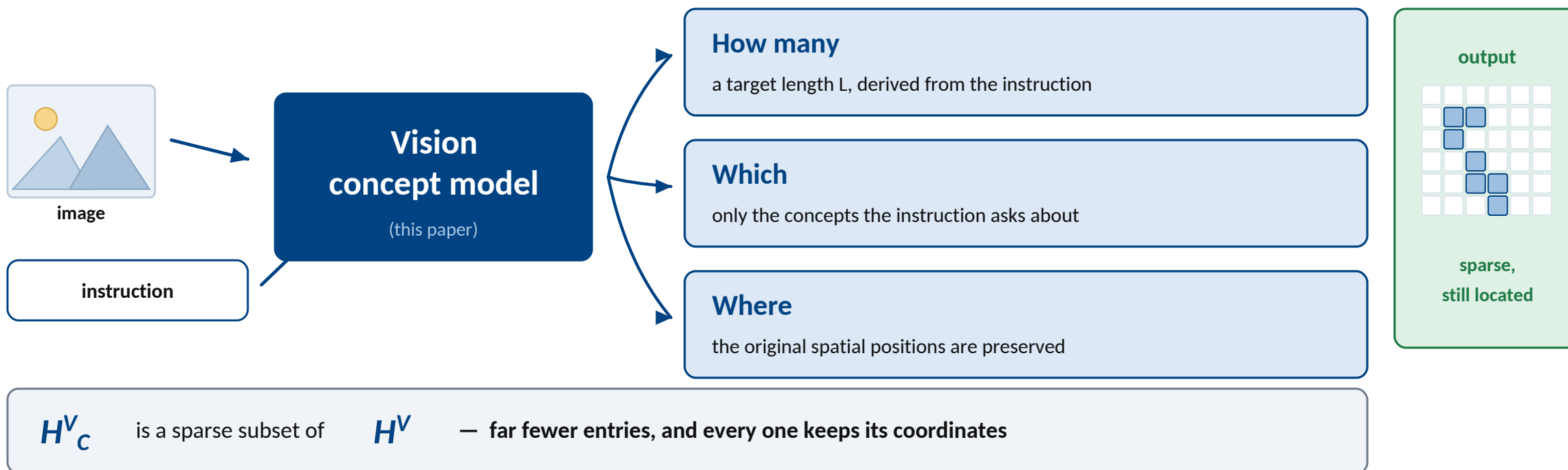
a fixed ratio cannot follow what  
the instruction actually asks for

**Consequence: applicable to visual question answering, and little else.**

Detection, segmentation and region-level tasks all need concepts that stay attached to the image.

**Shorter is not the same as conceptual — so none of them is a vision concept model.**

# Definition: the vision concept model



A model that decides **how many** concepts, **which** ones, and **where** — from the instruction.

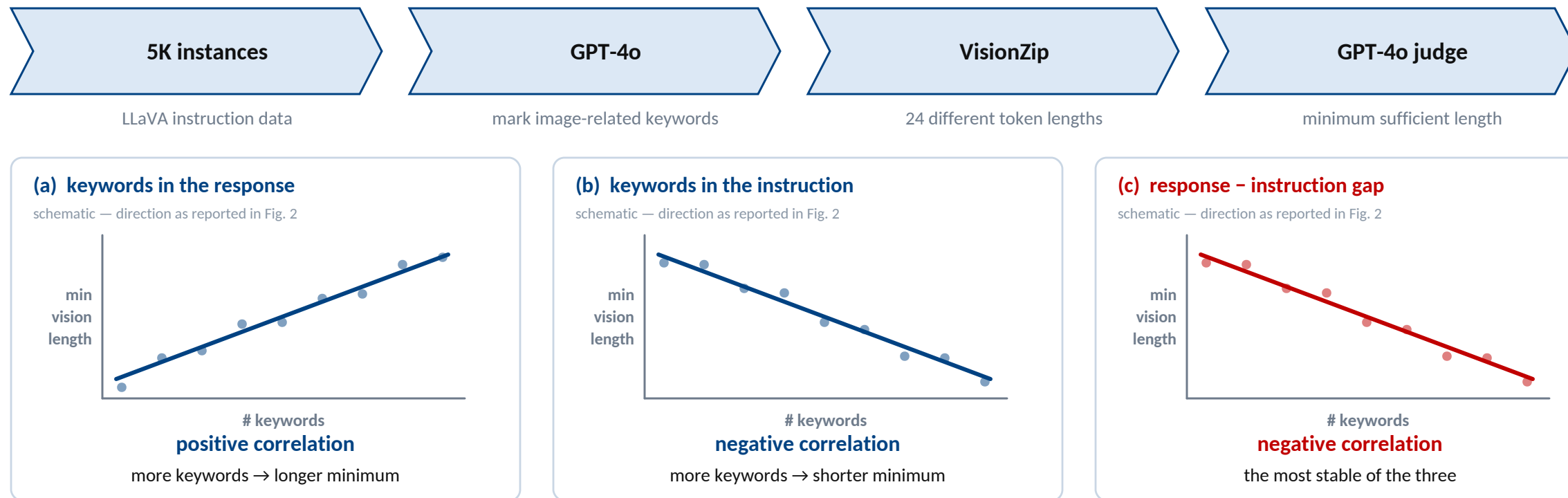
We need supervision **without labels**, and optimization **without a fixed length**.

**Needed: a signal already inside the data.**

**searching every cut point:  $O(2^M)$        $M = 576$**



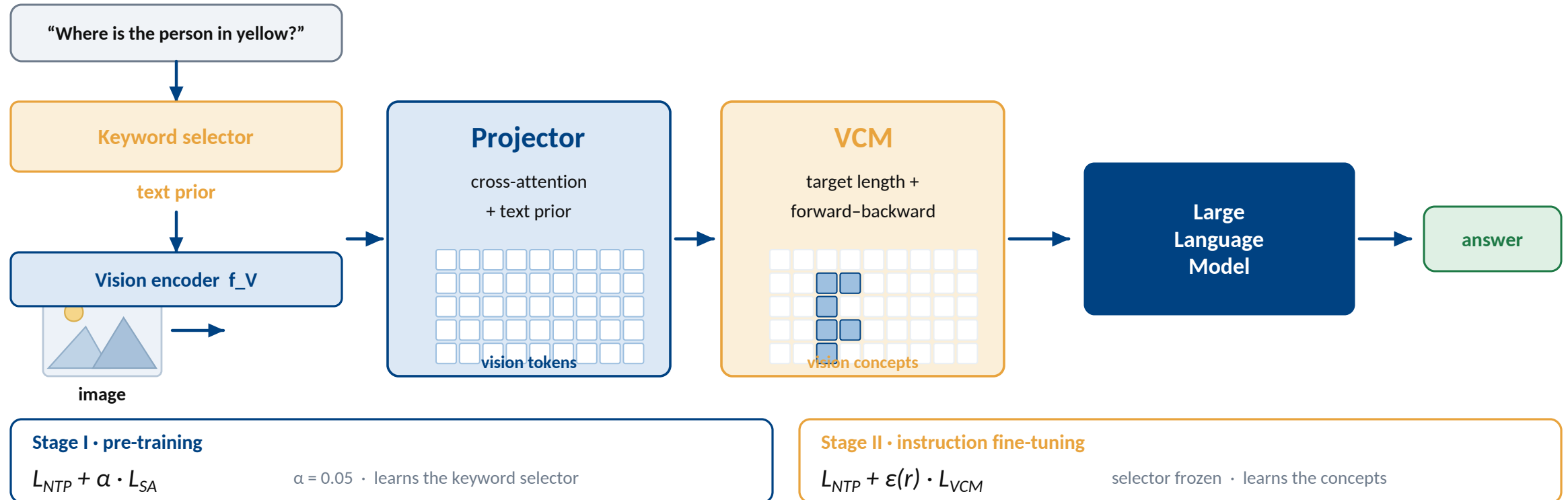
# The text prior already knows the answer



Probe: 5K LLaVA instances • GPT-4o extracts image-related keywords • VisionZip at 24 lengths • GPT-4o judges the minimum sufficient length.

The instruction-response **keyword gap** is a supervision signal that costs nothing.

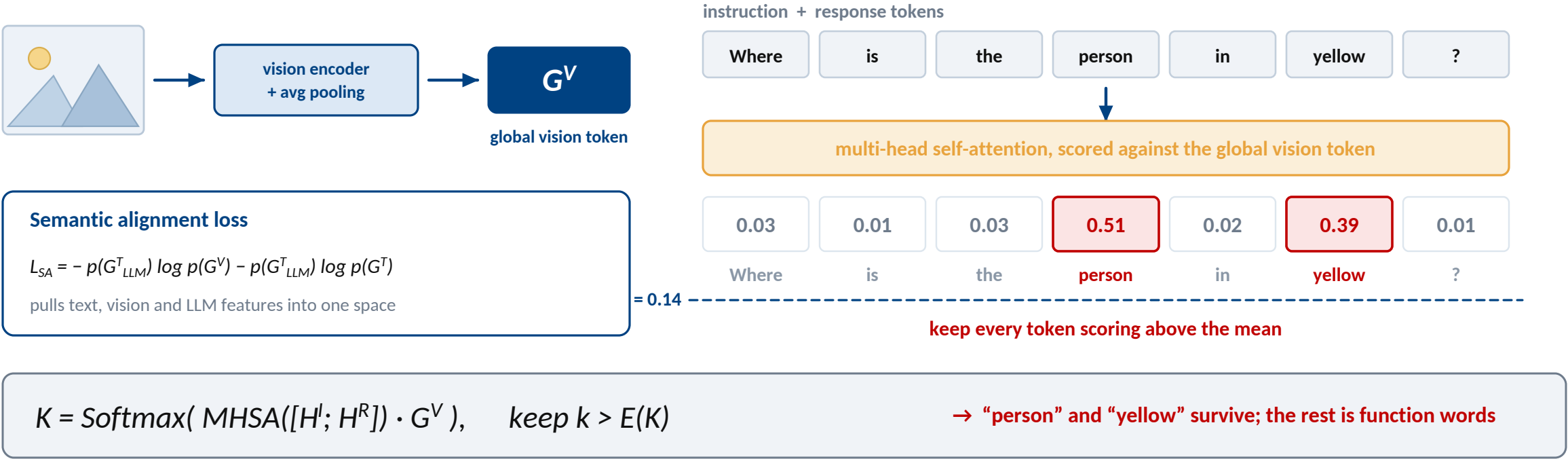
# VCM at a glance



Select keywords, estimate a length, then **align by dynamic programming**.

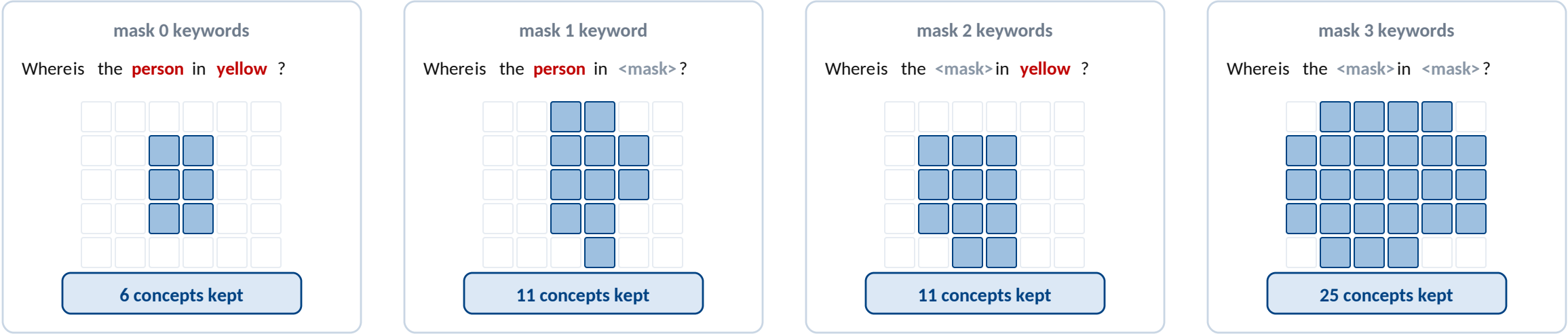
Figure 3(a) of the paper.

# Step 1 — keywords from semantic alignment



Keywords fall out of **vision-language alignment**, not out of human annotation.

# Step 2 — implicit contrastive sampling



more masking → more vision information required

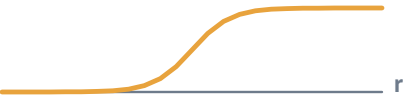
## Why this is safe

Masking never shortens the requirement, so the objective cannot learn to discard information the answer depends on.

$$r \sim \text{Uniform}(0, 1)$$

$$\epsilon(r) = a + (b-a) \cdot [1 + \tanh(k(2r-1))] / 2$$

$$a = 0.2, b = 1.2, k = 5$$



Masking keywords **lengthens** what is needed — a free contrast, and a **knob** at inference.

# Step 3 — turn the prior into a target length

$$L = \text{floor}( M \cdot S \cdot [ 1 - \text{Norm}(N_{\text{key}}) ] )$$

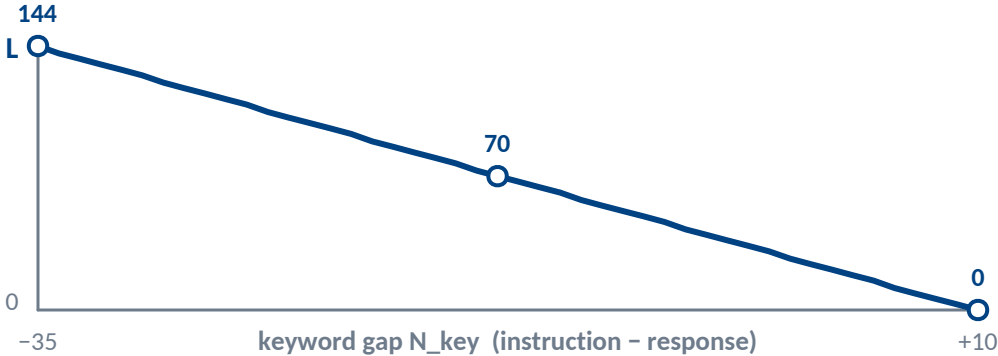
**M** input vision tokens    **S** information domain    **N\_key** keyword gap

## The information domain sets the budget

| S   | max tokens | Avg. (%) |
|-----|------------|----------|
| 1/2 | 288        | 58.9     |
| 1/4 | 144        | 59.3     |
| 1/6 | 72         | 56.4     |
| 1/8 | 36         | 57.2     |

Avg. over 4 VQA benchmarks (Table 4).

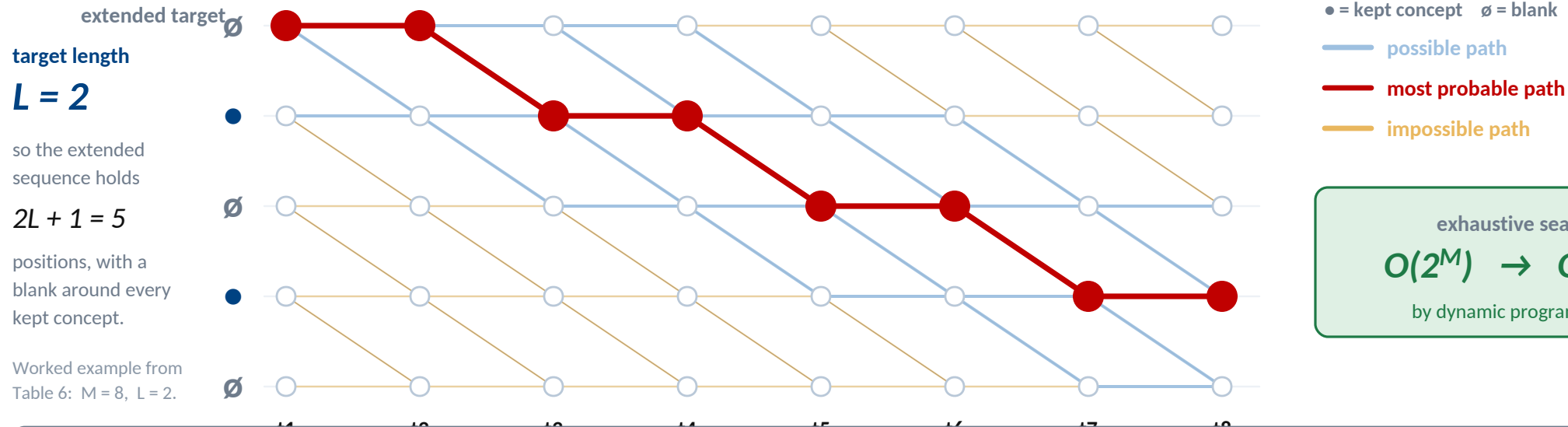
## The keyword gap moves the target length



Exact evaluation of the formula at M = 576; S = 1/4; N\_key is clipped to [-35, +10].

The text prior becomes an explicit **target length** — still no concept labels.

## Step 4 — forward-backward over all alignments



exhaustive search  
 $O(2^M) \rightarrow O(M^2)$   
by dynamic programming

$$\alpha(t, l) = p(z_l | y_t) \cdot [ \alpha(t-1, l) + \alpha(t-1, l-1) ]$$

$$\beta(t, l) = [ \beta(t+1, l) + \beta(t+1, l+1) ] \cdot p(z_l | y_t)$$

Every monotone path through the lattice is one way to cut the sequence into  $L$  concepts; the forward and backward recursions score all of them at once.

Where to cut is unknown, so we sum over **every** alignment:  $2^M \rightarrow M^2$ .

# The gradient is a posterior residual

$\gamma = \alpha\beta / p(Z|Y)$  — the posterior over alignments

|    | l=1   | l=2   | l=3   | l=4   | l=5   | row sum |
|----|-------|-------|-------|-------|-------|---------|
| t1 | 1.818 | 0.196 | 0.000 | 0.000 | 0.000 | 2.014   |
| t2 | 1.163 | 0.819 | 0.032 | 0.000 | 0.000 | 2.014   |
| t3 | 0.248 | 1.677 | 0.074 | 0.015 | 0.000 | 2.014   |
| t4 | 0.075 | 1.447 | 0.464 | 0.027 | 0.002 | 2.014   |
| t5 | 0.052 | 0.458 | 1.449 | 0.042 | 0.014 | 2.014   |
| t6 | 0.047 | 0.051 | 1.723 | 0.167 | 0.027 | 2.014   |
| t7 | 0.005 | 0.088 | 0.177 | 1.708 | 0.036 | 2.014   |
| t8 | 0.002 | 0.046 | 0.133 | 0.943 | 0.890 | 2.014   |

The row sum is constant in  $t$  — every column redistributes the same total probability mass.

From posterior to gradient

$$p(Z^V|Y^V) = \alpha(M, 2L) + \alpha(M, 2L+1) = \beta(1, 1) + \beta(1, 2)$$



$$L_{VCM} = -\log p(Z^V|Y^V)$$



$$\partial L_{VCM} / \partial y_t(z_l) = p(z_l|y_t) - \gamma(t, l)$$

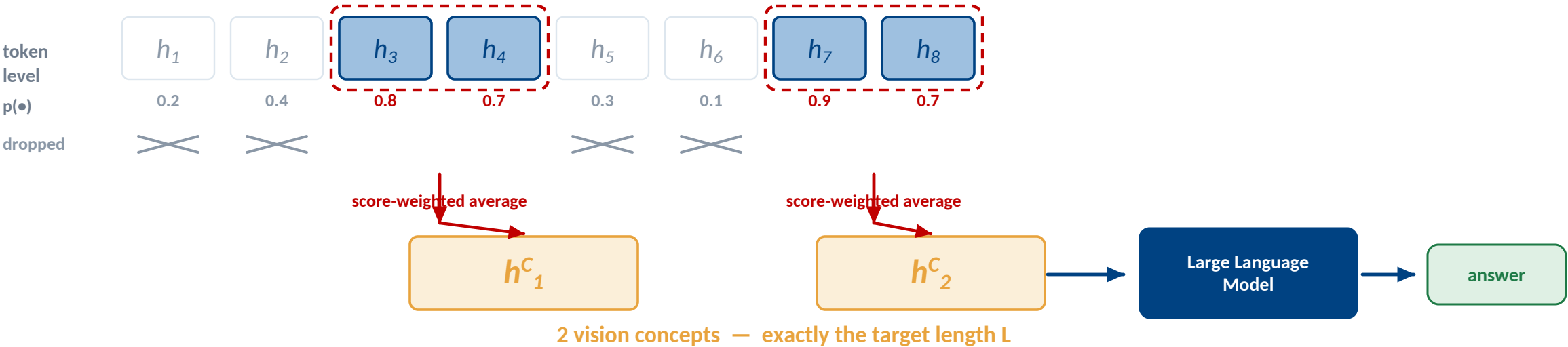
prediction minus posterior — the same shape as CTC.

keep token  $t$  when  $p(\bullet | y) > p(\emptyset | y)$

Worked example from Table 6:  $M = 8$  input tokens, target length  $L = 2$ . Row sums of  $\gamma$  stay constant across  $t$ .

One clean rule — prediction minus posterior, exactly CTC's gradient shape.

# Step 5 — merge the kept runs into concepts



### Implementation

Variable-length segments are merged with cumulative sums, not a double loop over batch and sequence.

100×  
faster

### The objective actually optimized

$$L^I = L_{NTP} + \alpha \cdot L_{SA}$$

$$L^{II} = L_{NTP} + \varepsilon(r) \cdot L_{VCM}$$

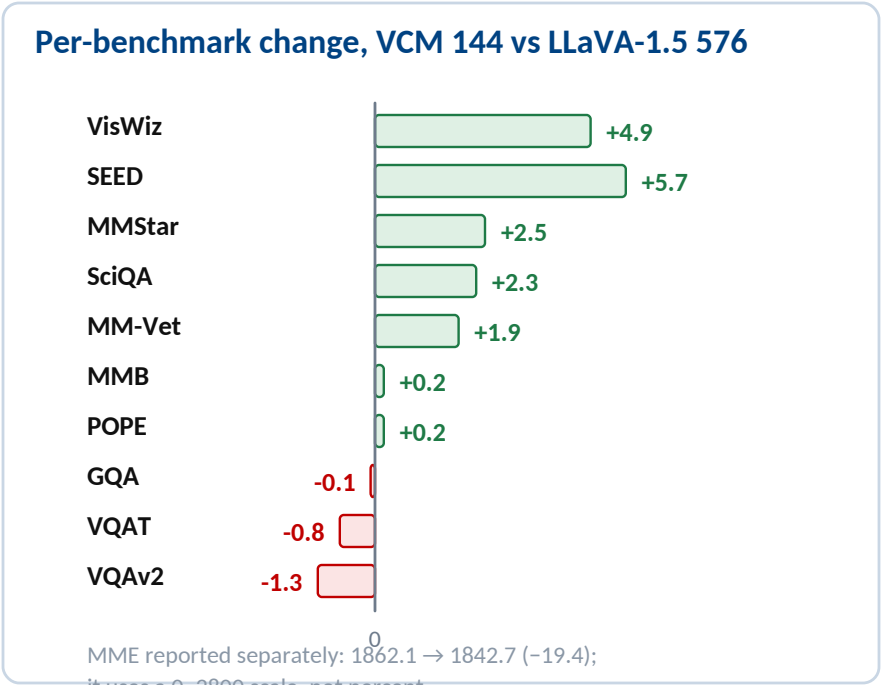
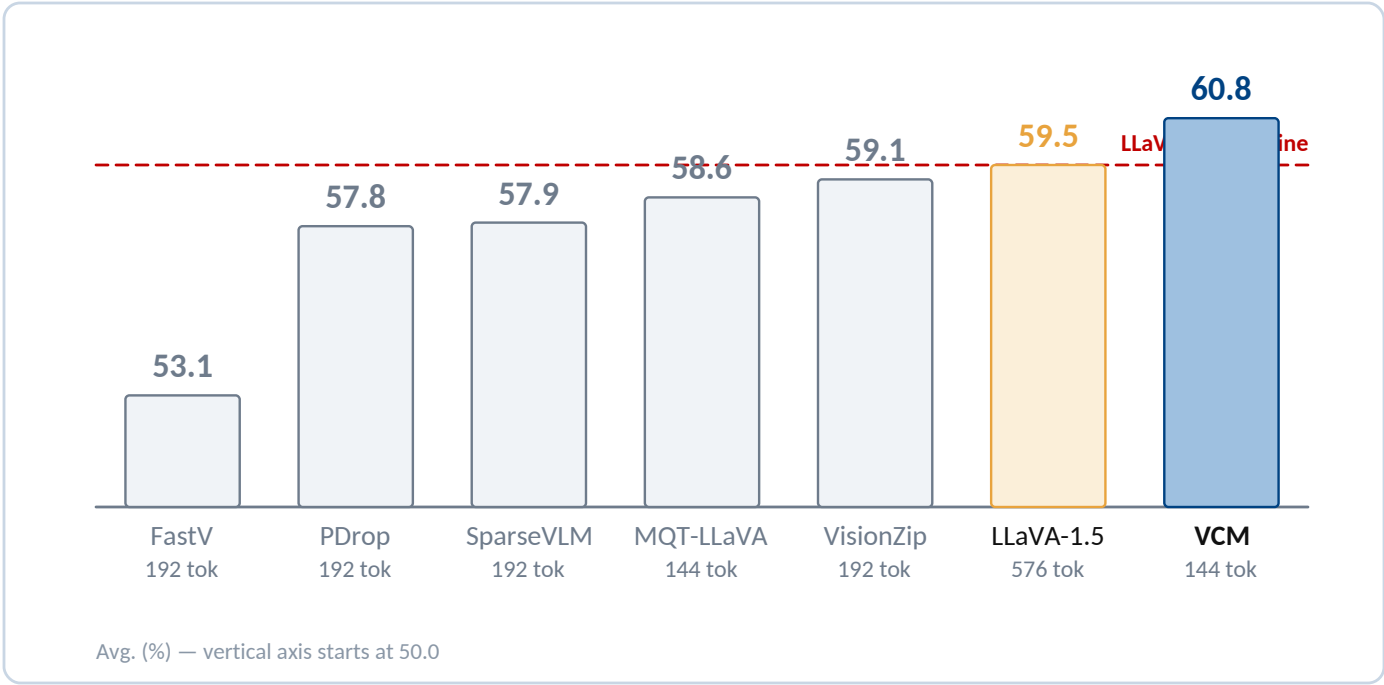
pre-training

instruction fine-tuning

Adjacent kept tokens merge by **score-weighted average** — and the op is **100× faster** than a double loop.



# Evidence — better than the model it compresses

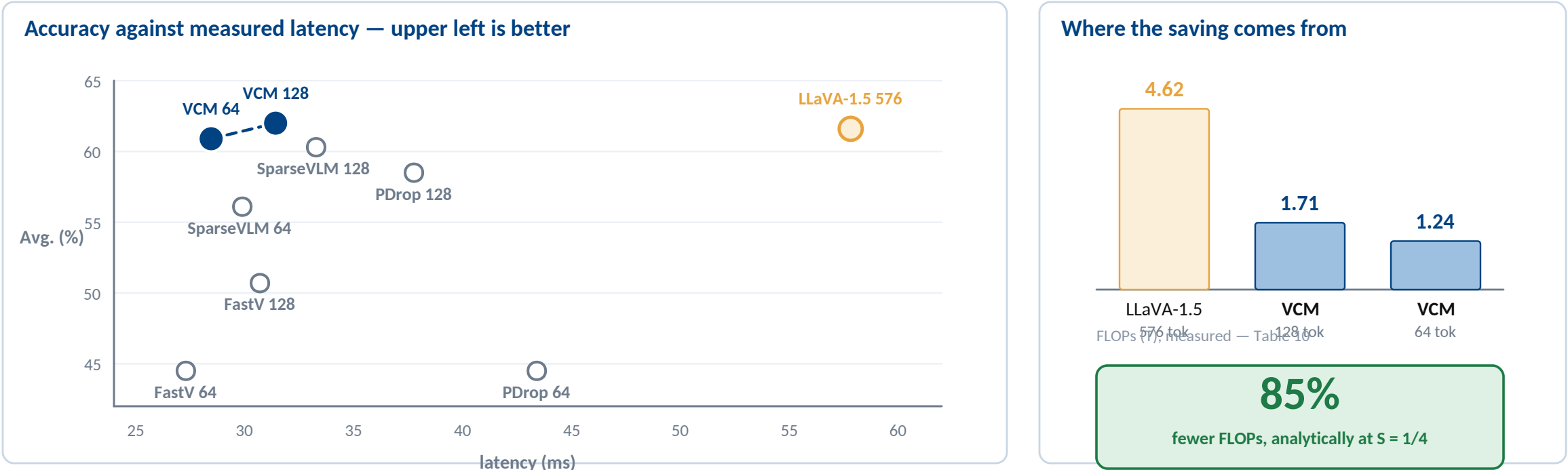


Average over 11 image-based VQA benchmarks (Table 1).

**144 tokens instead of 576, and a higher average than LLaVA-1.5 itself.**

Table 1 of the paper. #Vision Tokens = tokens fed to the LLM backbone.

# Evidence — the cost actually drops

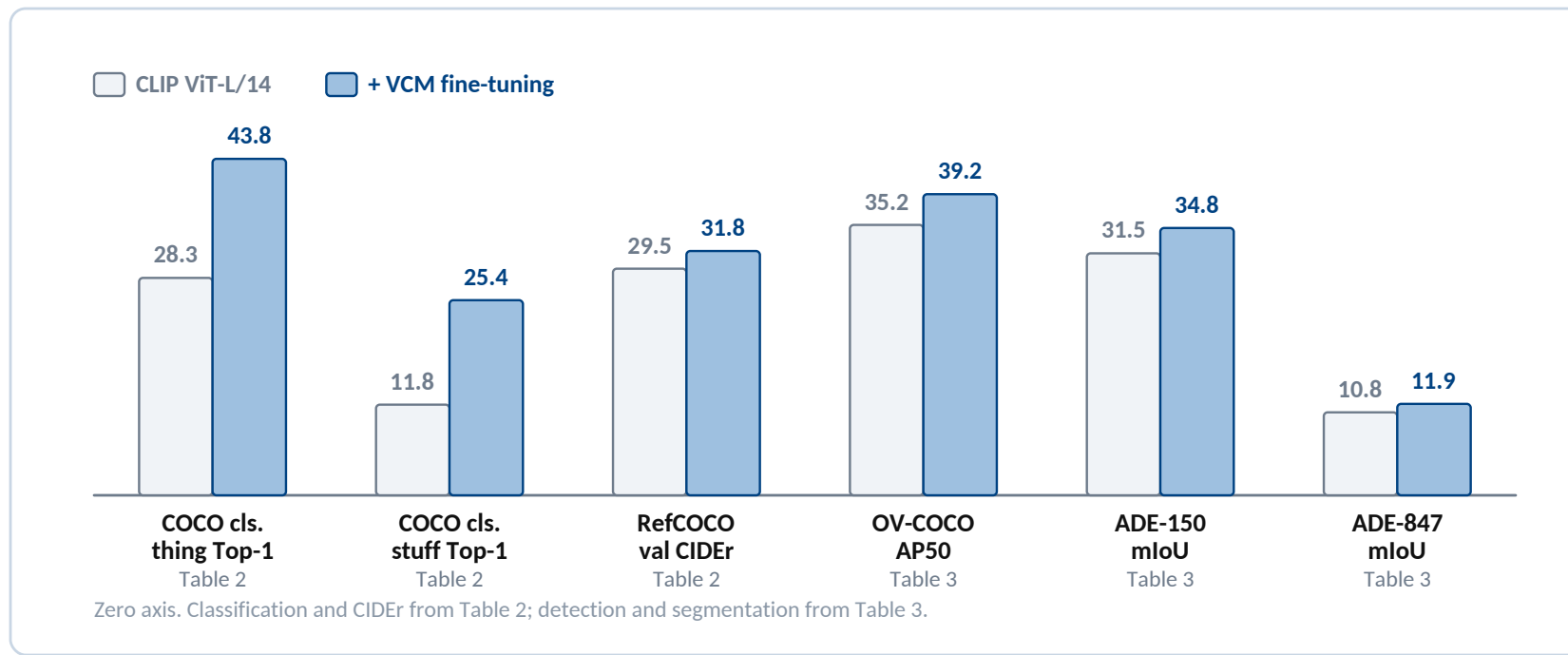


Average over the 8 benchmarks reported in Table 10; FLOPs and latency measured in the same setting.

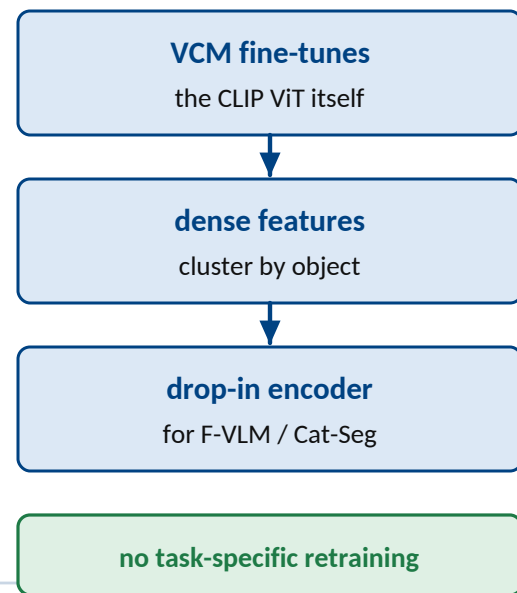
**85% fewer FLOPs in theory, 1.8× lower latency in practice, no average loss.**

Table 10 and Appendix D of the paper. Analytic ratio  $R \approx 3/25$  at  $S = 1/4$ .

# Evidence — the vision encoder itself improves



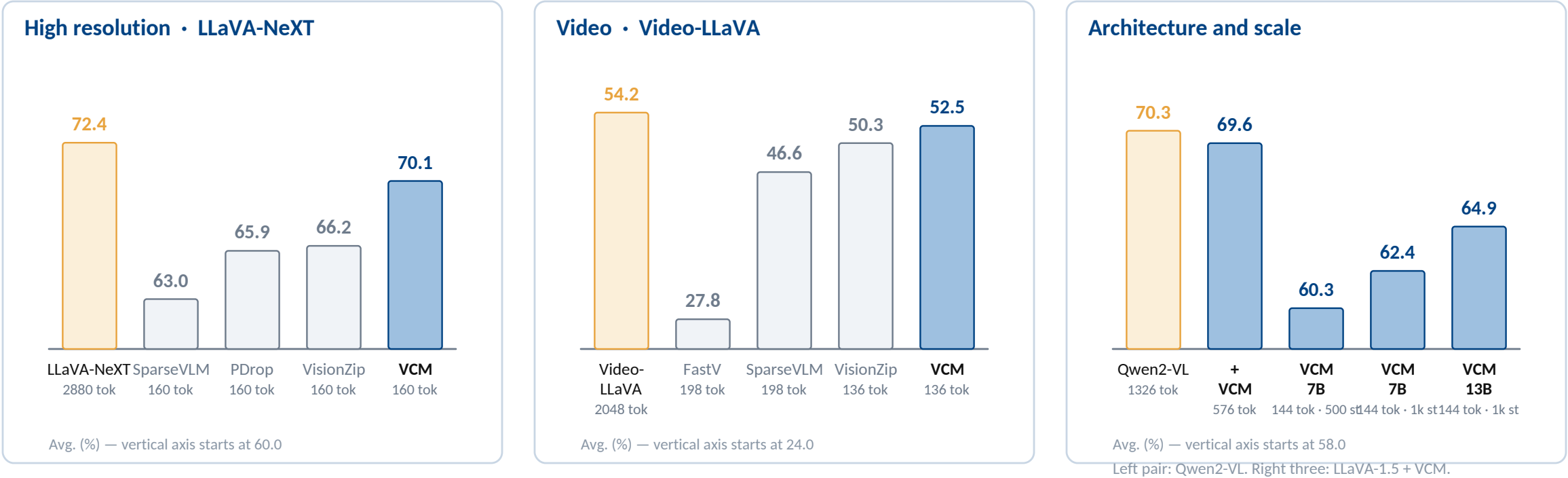
## Why it transfers



CLIP ViT-L/14 with and without VCM fine-tuning, dropped into F-VLM (detection) and Cat-Seg (segmentation).

**Not only compression — concept modeling improves dense perception.**

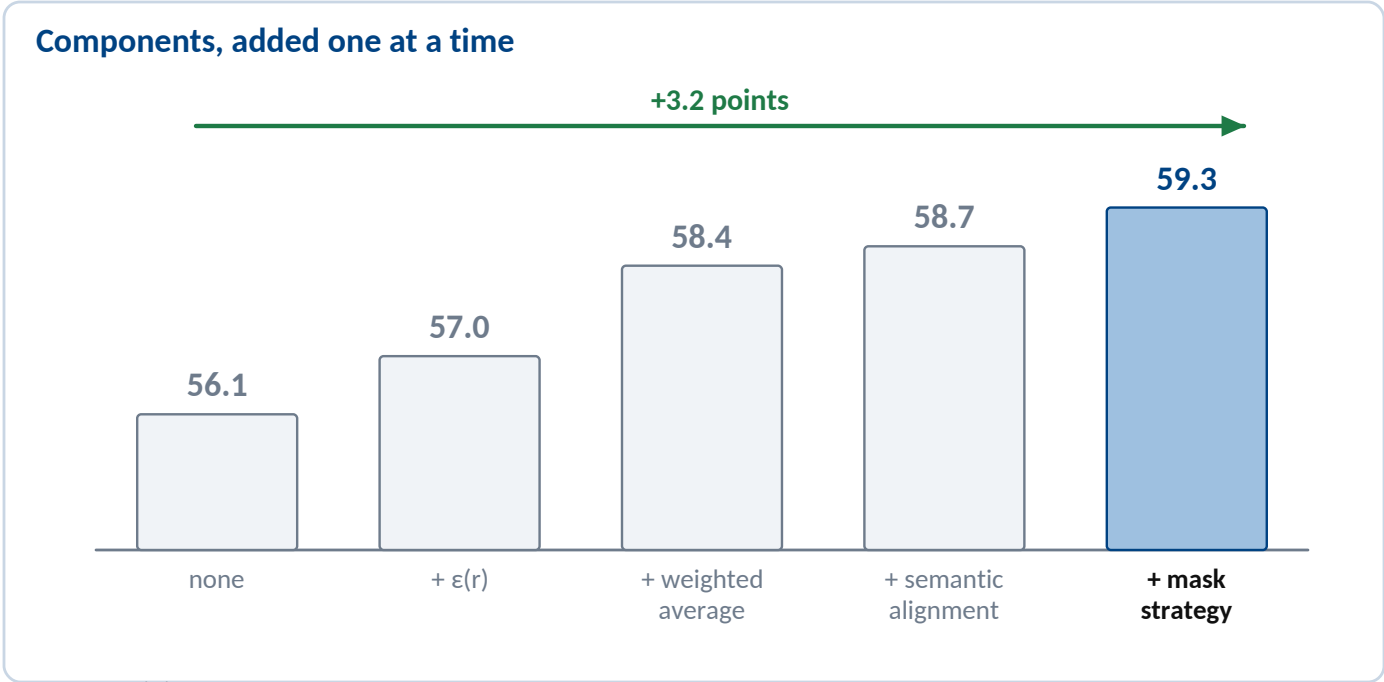
# Evidence — the recipe transfers



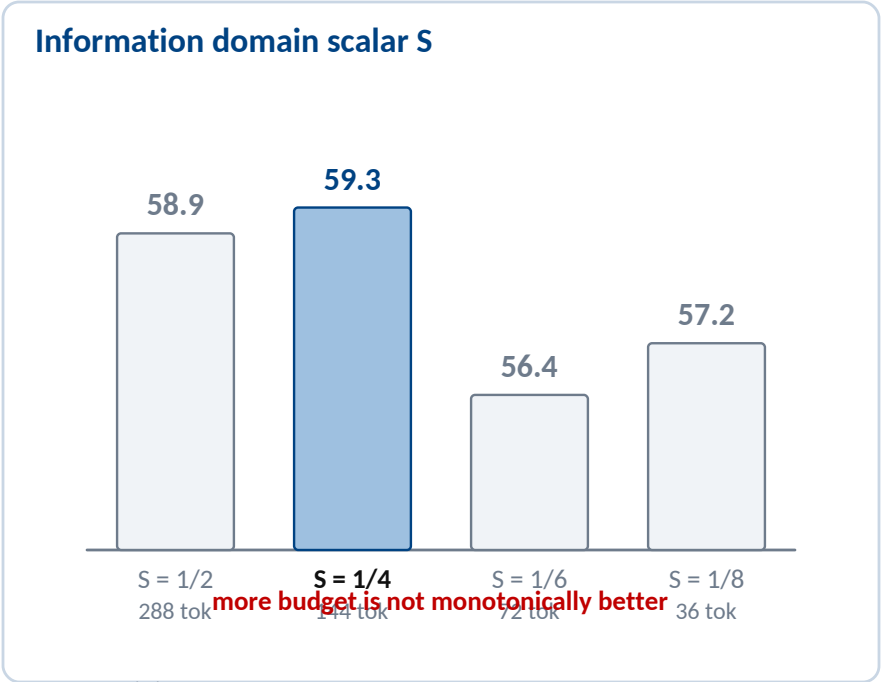
High resolution (Table 7) • video (Table 8) • other architectures and larger LLMs (Table 9).

Resolution, modality, architecture, scale — **the same recipe holds.**

# Ablation — what each piece buys



Avg. (%) — vertical axis starts at 54.0



Avg. (%) — vertical axis starts at 54.0

Average over 4 image-based VQA benchmarks, batch size 128, 500 training steps (Table 4).

Every component contributes, and  $S = 1/4$  is the accuracy–cost sweet spot.

# Conclusion

## Contributions

- 1 Define the vision concept model**  
quantity, identity and location, all conditioned on the instruction
- 2 A self-supervised framework**  
keyword selection plus a forward-backward algorithm; no concept labels
- 3 Efficiency and capability together**  
85% fewer FLOPs, and a stronger dense-perception encoder

## Limitations the paper states

### Keyword selection is adaptive

selected keywords may not be the true ones — a source of bias

### Length estimate is coarse

min-max normalization is a simplification of the mapping

Next: finer keyword selection with open-source LLMs.

**Model the concepts the instruction needs — not every token the image has.**

**Model the concepts the instruction **needs** — not every token the image **has**.**